

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Roly Steeven Cedeño Menéndez, Mtr.

Titulaciones	Semestre
• <i>Tecnologías de la Información en Línea</i>	<i>Quinto</i>

NOMBRES Y APELLIDOS: ANGEL PAUL VINCES CHAVEZ

PARALELO: A

SEMESTRE: QUINTO

ACTIVIDAD: ESTUDIO DE CASO – SISTEMA DE GESTIÓN DE INCIDENTES (HELP DESK)

PERIODO SEPTIEMBRE 2025 - ENERO 2026



Actividades a desarrollar en la Unidad _____

Actividad # _____

ACTIVIDAD #1: ESTUDIO DE CASO – SISTEMA DE GESTIÓN DE INCIDENTES (HELP DESK)

INTRODUCCIÓN

En la actualidad, las organizaciones dependen en gran medida de sistemas tecnológicos para realizar sus actividades diarias. Debido a esto, resulta indispensable contar con herramientas que permitan gestionar de forma eficiente incidentes técnicos, solicitudes de soporte y problemas reportados por los usuarios. Un sistema Help Desk tiene como finalidad registrar, clasificar, priorizar y dar seguimiento a incidentes tecnológicos dentro de una empresa u organización. Su correcta implementación mejora significativamente la atención al usuario, reduce tiempos de respuesta y optimiza los procesos internos del área de soporte técnico. En este estudio de caso se analiza el funcionamiento de un sistema de gestión de incidentes, identificando actores involucrados, procesos principales, diagramas UML y patrones de diseño aplicados para mejorar la estructura y eficiencia del sistema.

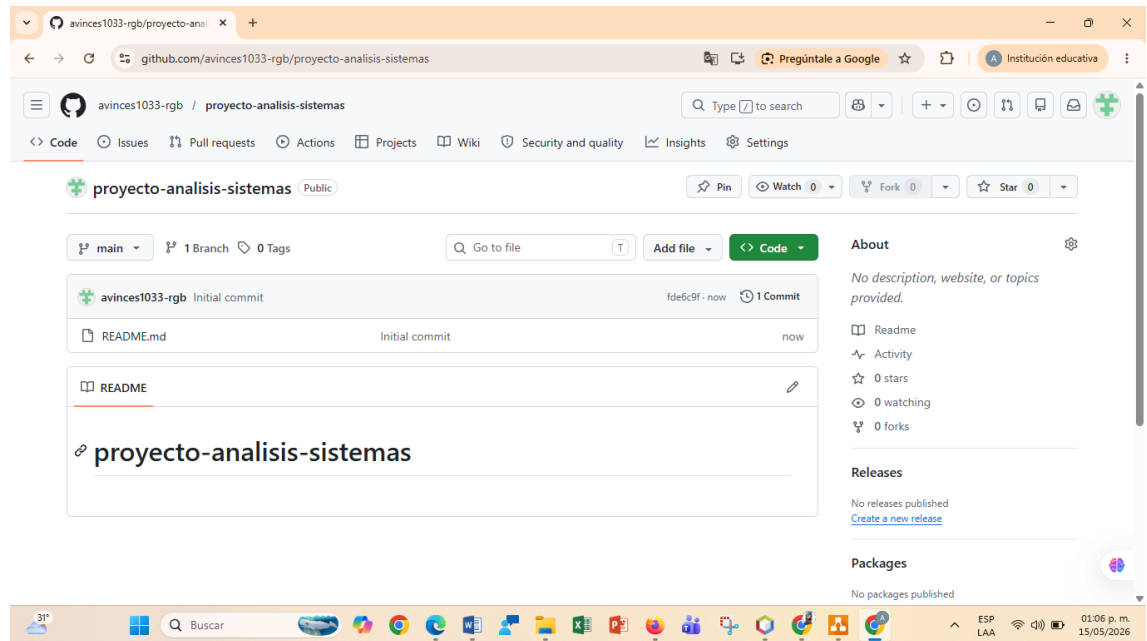
1. CREACIÓN DEL REPOSITORIO EN GITHUB

Como parte inicial del proyecto se creó un repositorio público en GitHub destinado al almacenamiento y control de versiones del trabajo realizado.

Dentro del repositorio se organizaron carpetas específicas para guardar documentación técnica, archivos markdown y diagramas UML generados durante el análisis del sistema.

El uso de GitHub permite:

- Mantener respaldo del proyecto.
- Controlar cambios realizados.
- Facilitar colaboración organizada.
- Tener acceso remoto al trabajo.



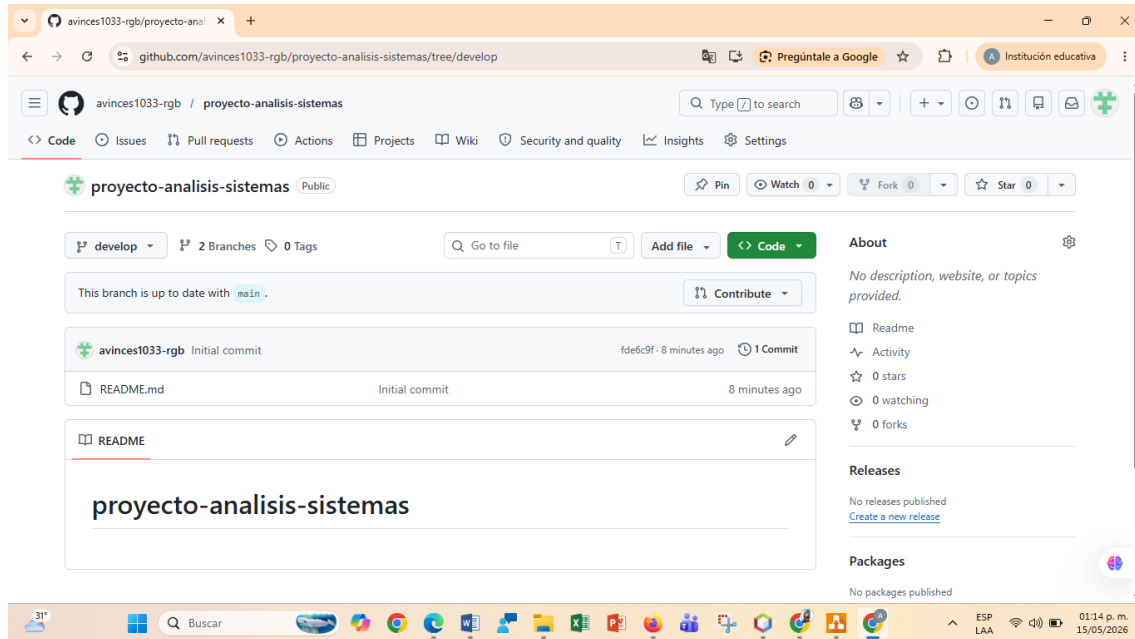
2. CREACIÓN DE LA RAMA DEVELOP CON GITFLOW

Se implementó la metodología Gitflow para una correcta organización del proyecto.

La rama **develop** fue creada para integrar avances generales del análisis sin afectar la rama principal del repositorio.

Beneficios:

- Separación entre código estable y desarrollo.
- Mejor organización del flujo de trabajo.
- Integración controlada de cambios.



3. CREACIÓN DE RAMAS FEATURE

Posteriormente se crearon ramas feature destinadas al desarrollo de tareas específicas.

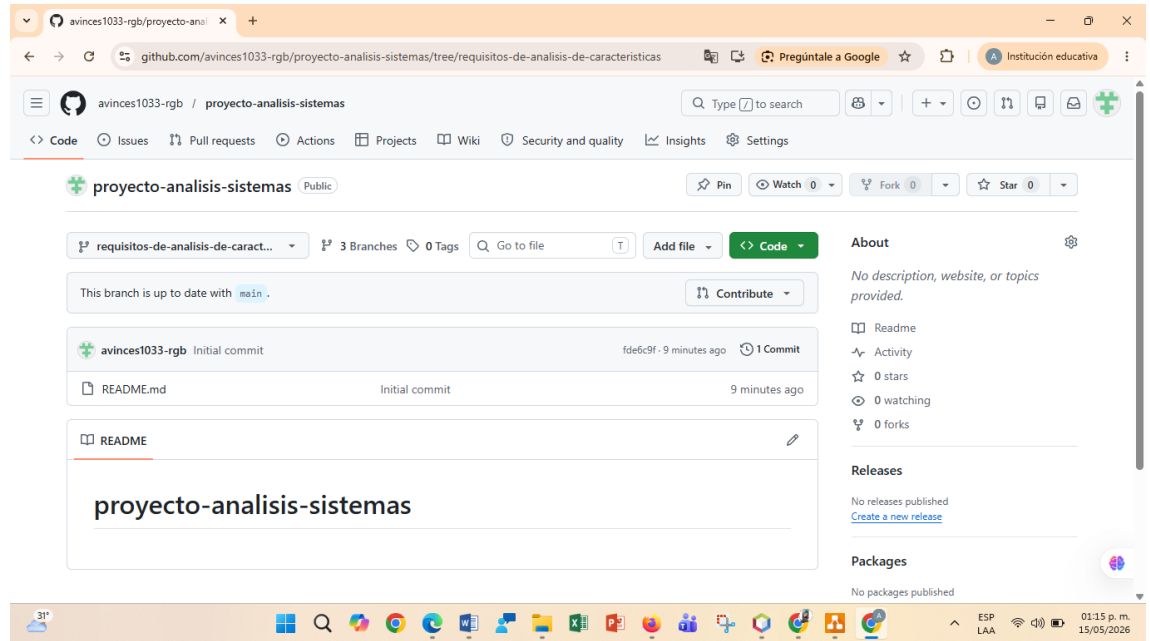
Ejemplo:

- feature/analisis-caracteristicas

Estas ramas permiten trabajar funcionalidades individuales sin comprometer otras partes del proyecto.

Ventajas:

- Mayor control.
- Menor riesgo de errores.
- Historial limpio.



4. DOCUMENTACIÓN DEL ANÁLISIS

Dentro de la rama feature se documentó el análisis del sistema Help Desk.

Se creó:

- Archivo analisis.md
- Carpeta docs
- Diagramas UML en formato PNG

Contenido documentado:

- Requisitos funcionales
- Requisitos no funcionales
- Diagramas de modelado



GitHub repository view for `proyecto-analisis-sistemas` in the `docs` directory, editing `analisis.md`.

Files: `requisitos-de-a...`, `README.md`

Content of `analisis.md`:

```
1 # Análisis del Escenario Técnico
2
3 ## 1. Actores del Sistema
4 - *Usuario Final*: Reporta incidentes en el Data Center.
5 - *Técnico de Soporte*: Atiende y resuelve los tickets.
6
7 ## 2. Requerimientos Funcionales
8 - RF1: El sistema debe permitir crear tickets de incidentes.
9 - RF2: El sistema debe notificar al técnico cuando se cree un ticket.
10
11 ## 3. Requerimientos No Funcionales
12 - RNF1: El sistema debe responder en menos de 3 segundos.
13
14
15 ## 4. Guarda el archivo
16 1. Baja hasta donde dice "Commit new file".
17 2. En "Commit message" escribe: Se agrega análisis del escenario técnico
18 3. Deja marcado: "Commit directly to the requisitos-de-analisis-de-caracteristicas branch"
```

Use `Control + Shift + M` to toggle the `tab` key moving focus. Alternatively, use `esc` then `tab` to move to the next interactive element on the page.

Attach files by dragging & dropping, selecting or pasting them.

GitHub repository view for `proyecto-analisis-sistemas` in the `docs` directory, showing the commit history for `analisis.md`.

Files: `requisitos-de-a...`, `docs` (selected), `analisis.md`, `README.md`

Commit history for `analisis.md`:

Name	Last commit message	Last commit date
..		
analisis.md	Create analisis.md	now



5. DIAGRAMA DE CASOS DE USO

El diagrama de casos de uso representa las interacciones entre actores y sistema.

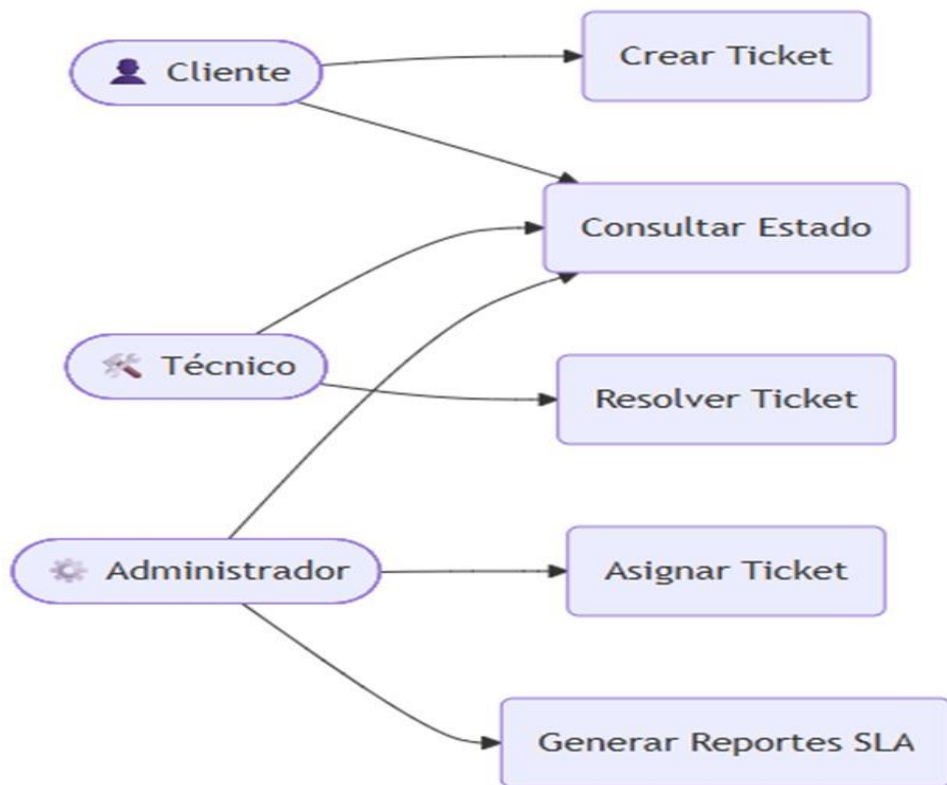
Actores identificados

- Usuario Final
- Soporte Técnico
- Administrador

Funciones principales

- Crear ticket
- Consultar ticket
- Autenticación de usuario
- Gestión de usuarios
- Reportes

Este diagrama permite comprender visualmente las responsabilidades de cada actor.



6. DIAGRAMA DE CLASES

El diagrama de clases modela la estructura del sistema orientado a objetos.

Clases principales

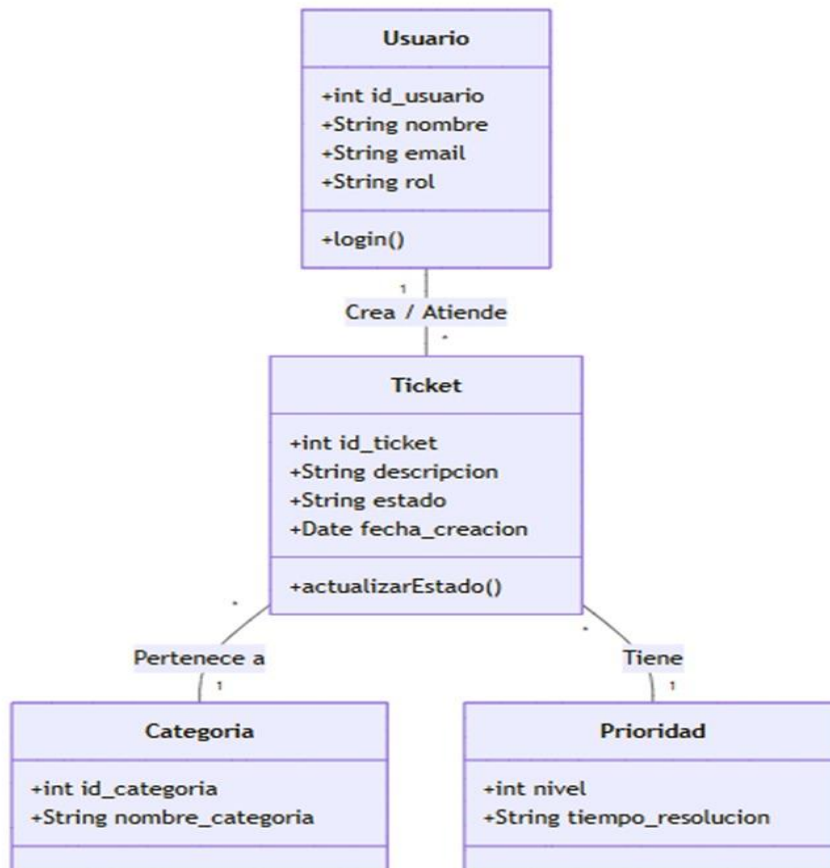
- Usuario
- Ticket
- Técnico
- Administrador
- Categoría
- Comentario

Relaciones



- Usuario crea Ticket
- Ticket pertenece a Categoría
- Ticket asignado a Técnico
- Ticket contiene Comentarios

Este modelo sirve como base para el diseño del sistema.



7. DIAGRAMA DE SECUENCIA

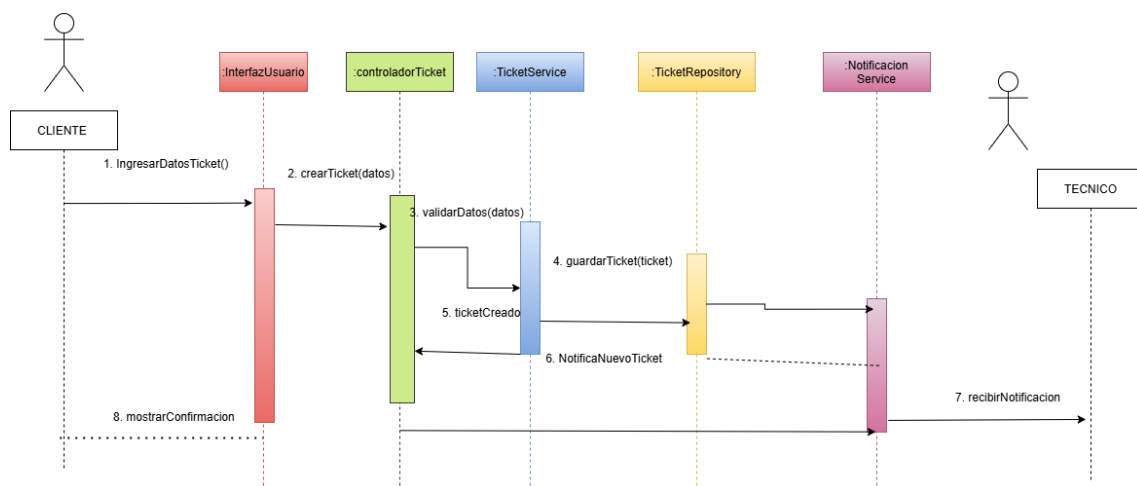
Representa el flujo temporal del proceso de gestión de incidentes.

Flujo principal

1. Usuario crea ticket
2. Sistema valida datos

3. Sistema confirma recepción
4. Notifica a soporte técnico
5. Técnico actualiza estado
6. Sistema guarda cambios
7. Usuario recibe actualización

Permite visualizar la interacción cronológica del proceso.



8. APLICACIÓN DE PATRONES DE DISEÑO

Durante el desarrollo se aplicaron patrones de diseño para mejorar estructura y mantenimiento.

Observer

Permite notificar automáticamente cambios a múltiples objetos dependientes.

Aplicación:

- Notificación cuando se crea un ticket.

Factory

Centraliza la creación de objetos.

Aplicación:



- Creación de diferentes tipos de tickets.

Singleton

Garantiza una sola instancia global.

Aplicación:

- Gestor de configuración o conexión.

Captura 8: Código de patrones implementados

CONCLUSIÓN

La implementación de un sistema de gestión de incidentes resulta fundamental para optimizar procesos de soporte técnico. Mediante este análisis se logró identificar componentes esenciales del sistema, modelar su estructura mediante diagramas UML y aplicar patrones de diseño orientados a buenas prácticas de ingeniería de software.

El uso de Gitflow, documentación organizada y control de versiones fortalece el desarrollo estructurado del proyecto y mejora significativamente su mantenibilidad y escalabilidad.

EVIDENCIAS

```
package sistema.gestor.eventos;
```

```
import java.util.ArrayList;
```

```
import java.util.List;
```

```
public class GestorEventos {
```

```
    private List<IObservadorEvento> suscriptores = new ArrayList<>();
```



```
public void suscribir(IObservadorEvento persona) {  
    suscriptores.add(persona);  
}  
  
public void avisarSuscriptores(String mensaje) {  
    for (IObservadorEvento p : suscriptores) {  
        p.recibirNotificacion(mensaje);  
    }  
}  
  
public void crearEvento(String tipo) {  
    System.out.println("\n=== CREANDO EVENTO ===");  
  
    Evento evento = EventoFactory.generarEvento(tipo);  
  
    if (evento != null) {  
        evento.describir();  
        avisarSuscriptores("Nuevo evento disponible: " + tipo);  
    } else {  
        System.out.println("Error: Tipo de evento no existe.");  
    }  
}  
  
public static void main(String[] args) {
```



```
GestorEventos gestor = new GestorEventos();

Asistente a1 = new Asistente("Maria");

Asistente a2 = new Asistente("Luis");


gestor.suscribir(a1);

gestor.suscribir(a2);


gestor.crearEvento("Concierto");

}
```



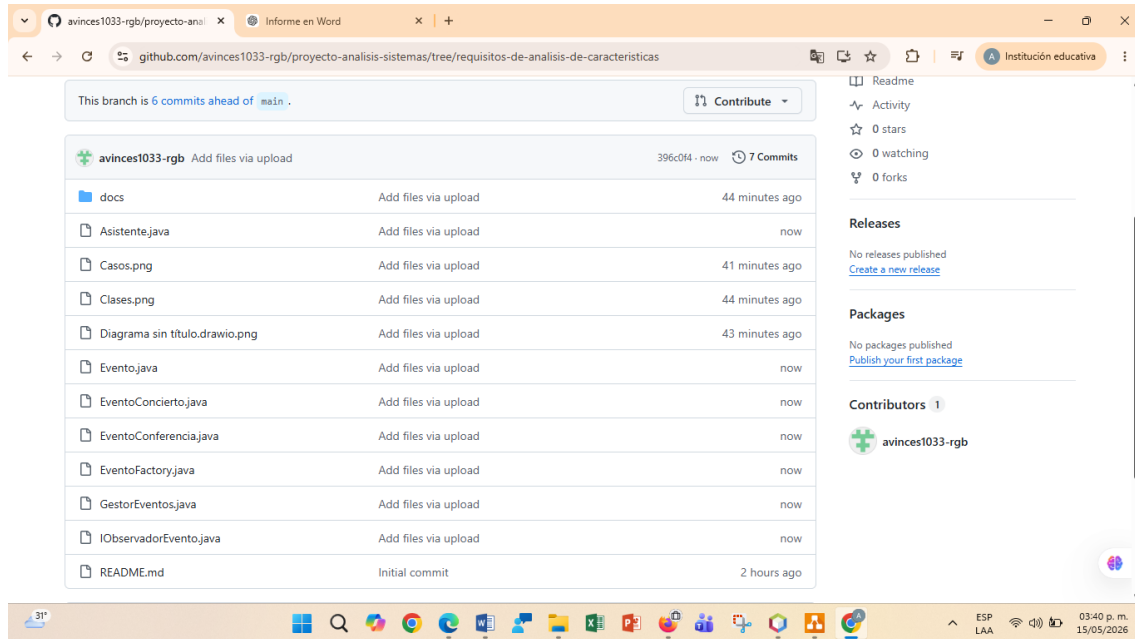
The image displays a development environment with two main components: a NetBeans IDE window and a web browser window showing a GitHub repository.

NetBeans IDE Window:

- Project Explorer:** Shows a project named "Sistema-gestor-eventos" with a package "sistema.gestor.eventos" containing files like "Asistente.java", "Evento.java", "EventoConcierto.java", "EventoFactory.java", "EventoGaleria.java", "GestorEventos.java", and "IObservadorEvento.java".
- Source Editor:** Displays the code for "GestorEventos.java". The code includes methods for "avisarSuscriptores", "crearEvento", and a "main" method. The "main" method calls "EventoFactory.generarEvento" and "GestorEventos.crearEvento".
- Output Window:** Shows the execution output, including the message "=== CREANDO EVENTO ===" and the details of the generated event: "Evento: Concierto de música en vivo programado. Asistente Maria: Nuevo evento disponible: Concierto Asistente Luis: Nuevo evento disponible: Concierto BUILD SUCCESSFUL (total time: 0 seconds)".

Web Browser Window:

- GitHub Repository:** The repository is named "proyecto-analisis-sistemas" and is owned by "avinces1033-rgb". It has 3 branches, 0 tags, and 6 commits.
- Files:** A list of files is shown, including "docs", "Casos.png", "Clases.png", "Diagrama sin titulo.drawio.png", and "README.md".
- README:** The README file is visible, showing the title "proyecto analisis sistemas".



LINK GITHUB

<https://github.com/avinces1033-rgb>

LINK DRIVE

https://drive.google.com/drive/folders/1VrvYrROKz_h56FbjXFYo-OS8MJrE8YYQ?usp=sharing

BIBLIOGRAFÍA

- GitHub. Plataforma de control de versiones y colaboración.
- Draw.io. Herramienta de diagramación UML.
- Documentación oficial Gitflow.
- Material de clase.